TECHNOLOGIES BIG DATA AVANCÉES · 2026

Pipeline de recherche sémantique sur documents (RAG)

Collecte, traitement distribué et recherche vectorielle sur Wikipédia en français

Groupe 6

Salma Abarkane · Cheffe de projet, architecture et intégration
Rania Bouaroua · Collecte (Airbyte)
Salma Benmouma · Orchestration (Prefect)
Oumaima Gassi · Stockage relationnel (PostgreSQL)
Imad Eddine El Khorassani · Stockage NoSQL (Qdrant)
Mohammed Yassine Benmeziane · Traitement distribué (Ray)
Anys Louzal · Visualisation et partage (Gradio)
Mohamed Anouar Faress · Couche RAG et évaluation

Stack : Airbyte · Prefect · PostgreSQL · Qdrant · Ray · Gradio
Dépôt : github.com/Anyslouzal/projet-rag-bigdata
Octobre 2026

Résumé

Ce projet met en place un pipeline Big Data complet qui transforme le corpus Wikipédia en français en un assistant de questions-réponses capable de citer ses sources. Il suit le principe du **RAG** (*Retrieval-Augmented Generation*) : on retrouve d'abord les passages pertinents dans une base vectorielle, puis un modèle de langage rédige la réponse à partir de ces seuls passages.

Les six technologies imposées couvrent chacune une étape : **Airbyte** collecte les fichiers parquet, **PostgreSQL** stocke la zone brute, les documents nettoyés et le suivi du pipeline, **Ray** découpe et vectorise les textes en parallèle, **Qdrant** indexe les vecteurs, **Prefect** orchestre l'ensemble et **Gradio** fournit l'interface. Le tout se lance avec Docker Compose.

Sur le fichier 16 du corpus, le pipeline a collecté 150 861 articles et en a retenu 112 313 après nettoyage. 2 000 articles ont été vectorisés en 14 290 passages. En optimisant le cluster Ray, le débit de vectorisation est passé de 2,9 à 28,1 passages par seconde. Sur 60 questions de test, la recherche retrouve l'article attendu en première position dans 86,7 % des cas et parmi les cinq premiers dans 93,3 % des cas, en 140 ms (médiane).

Sommaire

1. Introduction
2. Les données : Wikipédia en français
3. Architecture du pipeline
4. Mise en œuvre, brique par brique
5. Résultats et performances
6. Difficultés rencontrées et solutions
7. Limites et perspectives
8. Organisation de l'équipe
9. Conclusion

Annexes

1. Introduction

1.1 Contexte

Les grands modèles de langage savent rédiger, mais ils ne connaissent ni les documents privés d'une organisation ni les informations postérieures à leur entraînement, et ils peuvent inventer des faits. Le RAG répond à ce problème : avant de générer, on cherche dans une base documentaire les passages les plus proches de la question, et on demande au modèle de répondre uniquement à partir de ces passages, en les citant.

Dès que la base documentaire grossit, le RAG devient un problème Big Data. Il faut ingérer des millions de documents, les nettoyer, calculer un vecteur (*embedding*) pour chaque passage, ce qui est coûteux en calcul, puis indexer ces vecteurs pour les retrouver en quelques millisecondes. C'est ce que construit ce projet, sur un corpus réel : Wikipédia en français, soit environ 2,6 millions d'articles.

1.2 Objectifs

- Collecter un grand corpus de façon **incrémentale**, fichier par fichier.
- Nettoyer, dédoublonner et historiser les documents dans une base relationnelle.
- Découper et vectoriser les textes **en parallèle** sur un cluster de calcul.
- Indexer les vecteurs dans une base vectorielle interrogeable en temps réel.
- Orchestrer l'ensemble avec reprise sur erreur, planification et historique.
- Proposer une interface de questions-réponses qui cite ses sources.
- Mesurer les performances du traitement et la qualité de la recherche.

1.3 Technologies imposées

Brique	Technologie	Rôle dans le projet
Collecte	**Airbyte** (PyAirbyte)	Télécharge les fichiers parquet et les charge dans PostgreSQL
Orchestration	**Prefect 3**	Enchaîne les étapes, gère les reprises, la planification et l'historique
Stockage relationnel	**PostgreSQL 16**	Zone brute, documents nettoyés, métadonnées, suivi des runs et des questions
Stockage NoSQL	**Qdrant 1.20**	Base vectorielle : un point par passage (vecteur, texte, titre, lien)
Traitement	**Ray 2.59** (Ray Data)	Découpage et calcul des embeddings, distribués sur un cluster
Visualisation et partage	**Gradio 6**	Assistant, recherche sémantique et tableau de bord du pipeline

Pour la génération, nous utilisons le modèle ouvert **qwen2.5:7b** servi par **Ollama** sur la machine hôte, via une API compatible OpenAI. Changer de fournisseur (Mistral, Groq, OpenAI…) ne demande que trois variables d'environnement.

2. Les données : Wikipédia en français

2.1 Source

Nous utilisons le jeu de données `wikimedia/wikipedia` publié sur Hugging Face, dans sa version française du 1er novembre 2023 (`20231101.fr`). Il est découpé en **17 fichiers parquet** (environ 5 Go compressés) qui contiennent, pour chaque article, un identifiant, l'URL, le titre et le texte brut.

Ce corpus a quatre avantages pour le projet : il est en français, sous licence libre (CC BY-SA), il couvre des sujets très variés, et son volume réel (environ 2,6 millions d'articles) justifie une architecture distribuée. Le format parquet, en colonnes et compressé, se lit efficacement par lots.

2.2 Le fichier de travail

Pour le développement et les mesures, nous avons travaillé sur le **fichier 16** (environ 186 Mo). Le pipeline traite les autres fichiers de la même façon : il suffit de passer leurs numéros en paramètre.

Indicateur (fichier 16)	Valeur
Articles lus par Airbyte	150 861
Articles retenus après staging (≥ 500 caractères, sans doublon)	112 313 (74,4 %)
Volume de texte retenu	291,8 millions de caractères
Articles vectorisés pendant le projet	2 000
Passages (chunks) indexés dans Qdrant	14 290
Passages par article (moyenne)	7,1
Longueur moyenne / maximale d'un passage	395 / 500 caractères

Le filtre de 500 caractères écarte surtout les pages d'homonymie et les ébauches, qui apportent peu d'information à un assistant documentaire.

3. Architecture du pipeline

La figure 1 présente l'architecture. Les étapes de traitement (en blanc) lisent et écrivent dans des zones de stockage (en bleu foncé). Prefect orchestre l'ensemble, et le calcul intensif (en orange) est confié au cluster Ray.

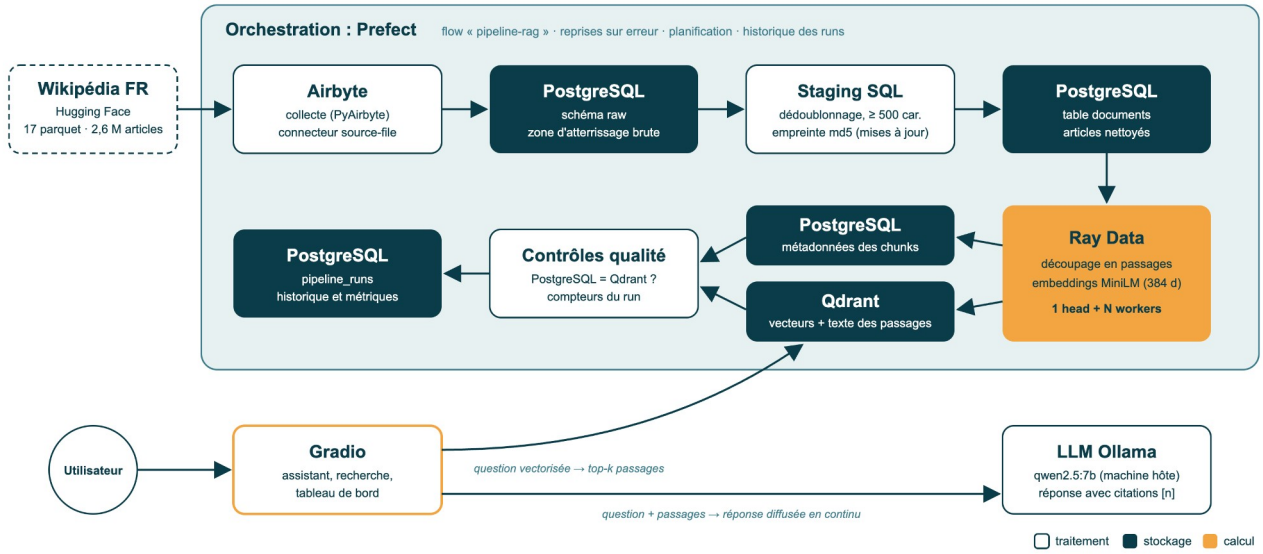


Figure 1 : *architecture du pipeline RAG*

3.1 Déroulé d'un run

1. **Collecte.** Airbyte lit un ou plusieurs fichiers parquet en HTTPS et les ajoute à la table `raw.wikipedia_fr`.
2. **Staging.** Une requête SQL dédoublonne les articles, écarte ceux de moins de 500 caractères et alimente la table `documents`. Un article modifié repasse en « à vectoriser » grâce à son empreinte md5. La zone brute est ensuite vidée.
3. **Vectorisation.** Prefect soumet un job au cluster Ray. Ray lit les documents par plages, les découpe en passages, calcule les embeddings, puis écrit les vecteurs dans Qdrant et les métadonnées dans PostgreSQL.
4. **Contrôles.** On vérifie que PostgreSQL et Qdrant contiennent le même nombre de passages, et le run est enregistré avec ses compteurs dans `pipeline_runs`.

3.2 Principes de conception

- **Idempotence.** Chaque passage a un identifiant déterministe (UUID v5 de `doc_id:index`). Relancer un run réécrit les mêmes points au lieu de créer des doublons.
- **Traitement incrémental.** Seuls les documents non encore vectorisés (`indexed_at IS NULL`) sont traités. Un run reprend là où le précédent s'est arrêté.
- **Séparation brut / nettoyé.** La zone `raw` reçoit les données telles quelles, la table `documents` ne contient que des données validées.
- **Observabilité.** Chaque run et chaque question posée sont tracés dans PostgreSQL, et chaque outil garde sa propre interface de suivi (Prefect, Ray, Qdrant).

3.3 Déploiement

Tout le projet se lance avec **Docker Compose**. Les services Python partagent une seule image (`ragpipe:dev`, Python 3.12) dont les versions sont figées dans `requirements.lock` : toute l'équipe exécute exactement le même code. Le LLM tourne sur la machine hôte, ce qui lui donne accès à l'accélération matérielle du Mac.

Service	Rôle	Port
`postgres`	PostgreSQL 16 : données du projet et base interne de Prefect	5432
`qdrant`	Base vectorielle et son tableau de bord	6333
`prefect-server`	Serveur et interface Prefect	4200
`ray-head`	Nœud principal Ray : ordonnanceur, API Jobs, tableau de bord	8265
`ray-worker` (×N)	Nœuds de calcul Ray, nombre réglable par `RAY_WORKERS`	—
`pipeline`	Déploiement Prefect qui exécute le flow (planification nocturne)	—
`app`	Application Gradio	7860

4. Mise en œuvre, brique par brique

4.1 Collecte : Airbyte

Nous utilisons **PyAirbyte**, la bibliothèque Python d'Airbyte, avec le connecteur officiel `source-file` (format parquet, fournisseur HTTPS). Les données sont écrites par le cache PostgreSQL d'Airbyte dans le schéma `raw`, en mode ajout (`append`). La liste des fichiers à collecter est un paramètre du flow Prefect.

Le connecteur dépend d'une bibliothèque incompatible avec Python 3.12. Airbyte l'installe donc dans un environnement Python 3.11 isolé (`use_python="3.11"`), préinstallé dans l'image Docker pour que la première collecte ne télécharge rien. La collecte et le staging du fichier 16 prennent environ **2 minutes**.

4.2 Stockage relationnel : PostgreSQL

PostgreSQL joue trois rôles : zone d'atterrissage, référentiel des documents et journal du pipeline. Le schéma est créé automatiquement au premier démarrage.

Table	Contenu
`raw.wikipedia_fr`	Données brutes écrites par Airbyte, vidées après chaque staging
`documents`	Articles nettoyés : titre, URL, texte, taille, empreinte md5, dates d'ingestion et de vectorisation. Clé unique (source, identifiant) et index partiel sur les documents à vectoriser
`chunks`	Métadonnées des passages : identifiant, document, rang, taille, modèle d'embedding
`pipeline_runs`	Un enregistrement par run : statut, fichiers, compteurs, durée, erreur
`query_logs`	Questions posées dans l'application : scores, latences, sources (JSONB)

Le **staging** est une seule requête SQL. `DISTINCT ON` garde un exemplaire par article, le filtre de longueur écarte les textes trop courts, et `INSERT … ON CONFLICT DO UPDATE … WHERE checksum <> …` n'écrase un document que si son contenu a changé. La colonne système `xmax` permet de distinguer les insertions des mises à jour : les anciens passages des documents modifiés sont supprimés de PostgreSQL et de Qdrant, puis revectorisés au run suivant.

4.3 Traitement distribué : Ray

Le cluster comprend un nœud **head**, qui ne fait que coordonner (`--num-cpus=0`), et **N workers** de 3 CPU et 2 Go chacun. Prefect soumet le traitement via l'API Ray Jobs. Le job est écrit avec **Ray Data**, qui exécute les étapes en flux : les premiers passages sont écrits dans Qdrant pendant que les suivants sont encore en cours de calcul.

Étape Ray Data	Ressources	Ce qu'elle fait
`from_items`	—	Découpe le travail en tâches de 200 identifiants de documents
`flat_map(load_and_chunk)`	0,5 CPU par tâche	Lit les documents dans PostgreSQL et les découpe en passages
`map_batches(EmbedChunks)`	1 acteur de 2 CPU par worker	Calcule les embeddings ; le modèle est chargé une seule fois par acteur
`map_batches(WriteChunks)`	1 acteur	Écrit les vecteurs dans Qdrant et les métadonnées dans PostgreSQL

Le **découpage** regroupe les paragraphes jusqu'à 500 caractères, avec 75 caractères de recouvrement, et coupe les paragraphes trop longs aux fins de phrase. Le titre de l'article est ajouté devant chaque passage avant le calcul du vecteur, car un passage isolé perd souvent son sujet.

Le modèle d'embedding est `paraphrase-multilingual-MiniLM-L12-v2`, exécuté sur CPU avec **FastEmbed** (ONNX). Il est multilingue, rapide et produit des vecteurs de 384 dimensions. Comme il ne lit que les 128 premiers tokens d'un texte, la taille des passages a été fixée à 500 caractères (voir partie 6). Le même modèle vectorise les passages et les questions.

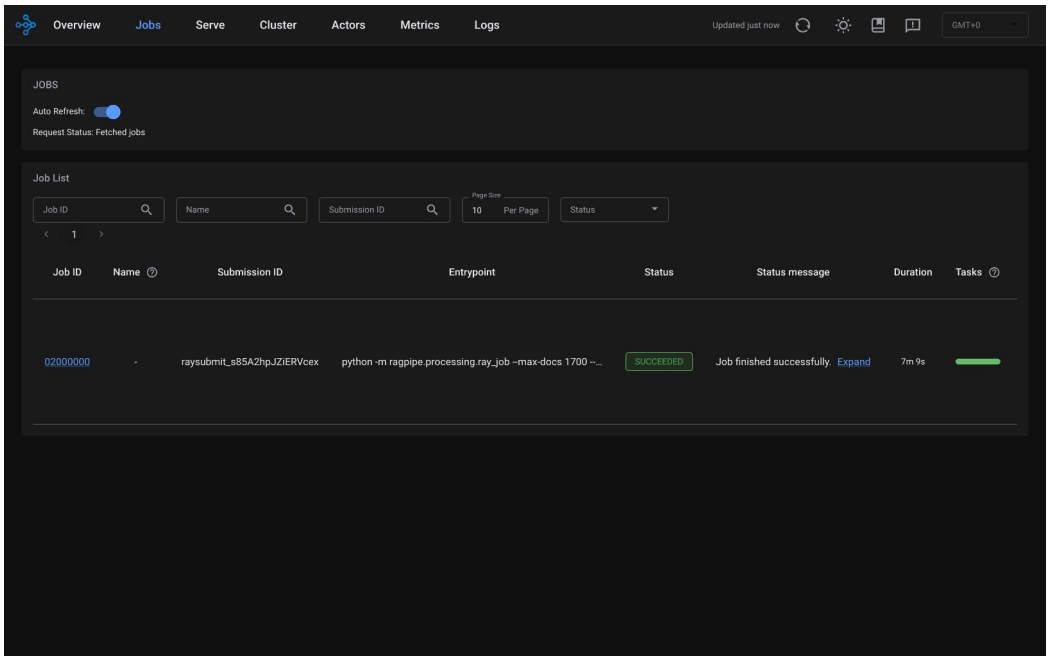


Figure 2 : job de vectorisation terminé dans le tableau de bord Ray

4.4 Stockage NoSQL : Qdrant

Chaque passage devient un **point** Qdrant : un vecteur de 384 dimensions et une charge utile (`doc_id`, rang, titre, URL, texte). La collection `wikipedia_fr` utilise la distance cosinus. Les vecteurs d'origine sont stockés sur disque, et une copie quantifiée en entiers 8 bits (int8) reste en mémoire : elle divise la mémoire par quatre et accélère la recherche, et les meilleurs candidats sont ensuite recalculés avec les vecteurs exacts (*rescoring*).

Deux index de charge utile complètent la recherche vectorielle : un index entier sur `doc_id`, pour supprimer rapidement les passages d'un article modifié, et un index plein texte sur le titre, utilisé par le filtre de l'onglet Recherche.

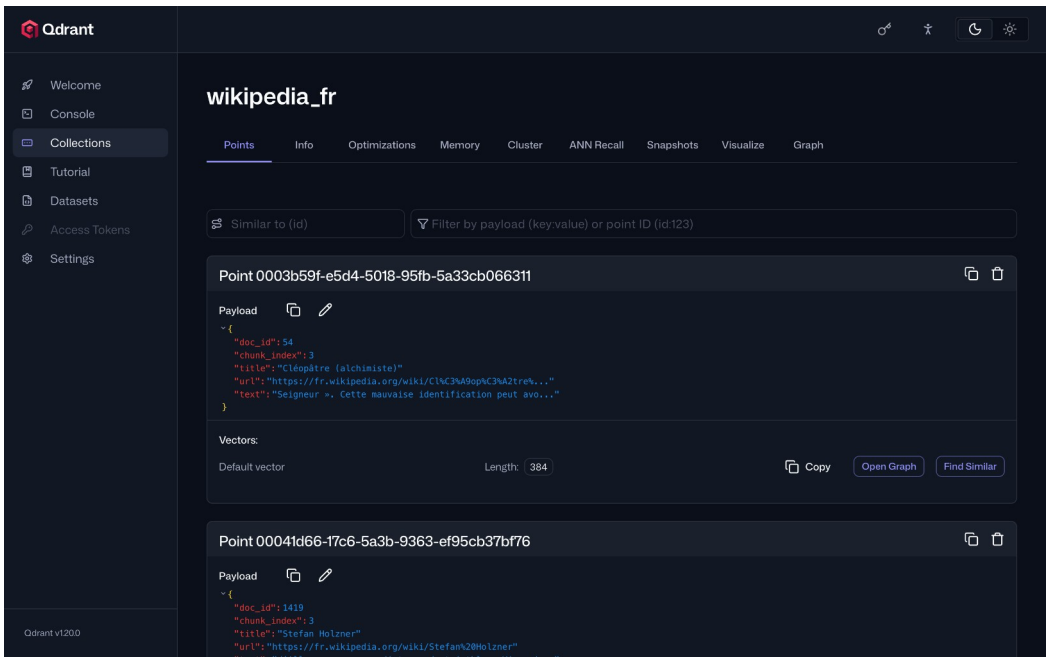


Figure 3 : un point de la collection wikipedia_fr dans le tableau de bord Qdrant

4.5 Orchestration : Prefect

Le flow `pipeline-rag` enchaîne quatre tâches : `collecte-airbyte` (2 reprises automatiques, à 60 s d'intervalle), `staging-postgres`, `vectorisation-ray` et `controles-qualite`. La tâche de vectorisation soumet le job Ray, suit son état, puis lit son résultat dans les logs. Chaque run est aussi enregistré dans `pipeline_runs` avec ses compteurs, ou l'erreur en cas d'échec.

Le conteneur `pipeline` publie le déploiement `wikipedia-fr`, planifié chaque nuit à 2 h (planification en pause par défaut). Depuis l'interface Prefect, on peut lancer un run en modifiant ses paramètres : fichiers à collecter, nombre de documents à vectoriser, nombre d'acteurs d'embedding, collecte ou non. Le serveur Prefect stocke son état dans la même instance PostgreSQL, dans une base séparée.

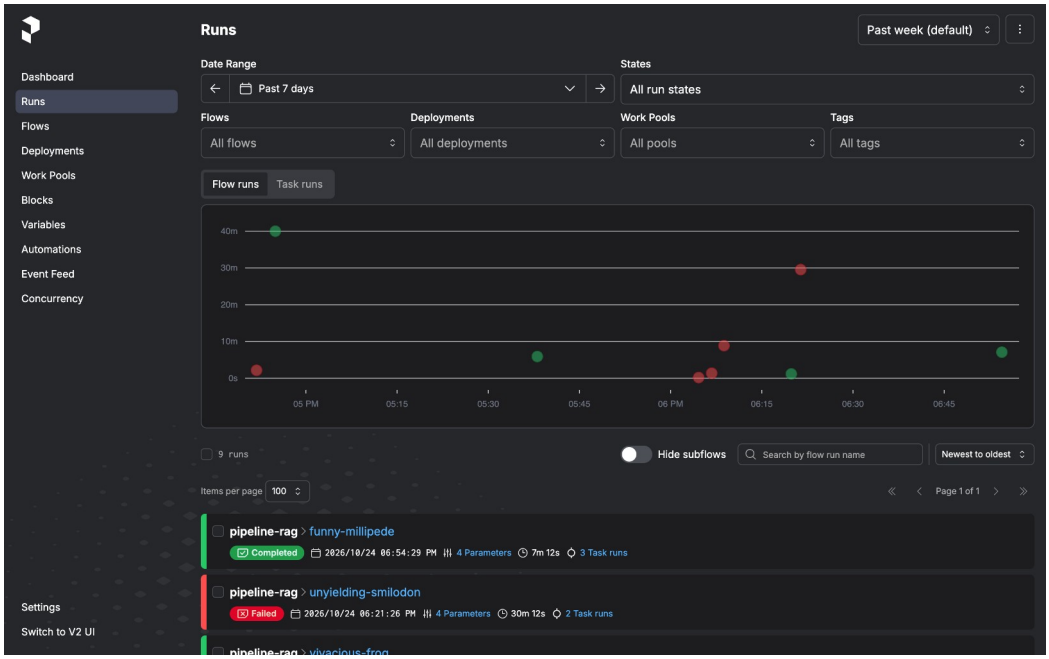


Figure 4 : historique des runs dans l'interface Prefect

4.6 Visualisation et partage : Gradio

L'application Gradio comporte trois onglets :

- **Assistant** : conversation avec réponse diffusée au fil de l'eau, choix du nombre de passages utilisés (top-k) et affichage des sources avec leur score de similarité et un lien vers l'article.
- **Recherche sémantique** : les passages les plus proches d'une requête, avec un filtre sur le titre.
- **Pipeline** : volumes du corpus, derniers runs et dernières questions, lus dans PostgreSQL.

Chaque question posée est enregistrée avec la durée de la recherche, celle de la génération, le meilleur score et les sources, ce qui permet de suivre l'usage et la qualité dans le temps.



Figure 5 : onglet Assistant : réponse citée et passages utilisés

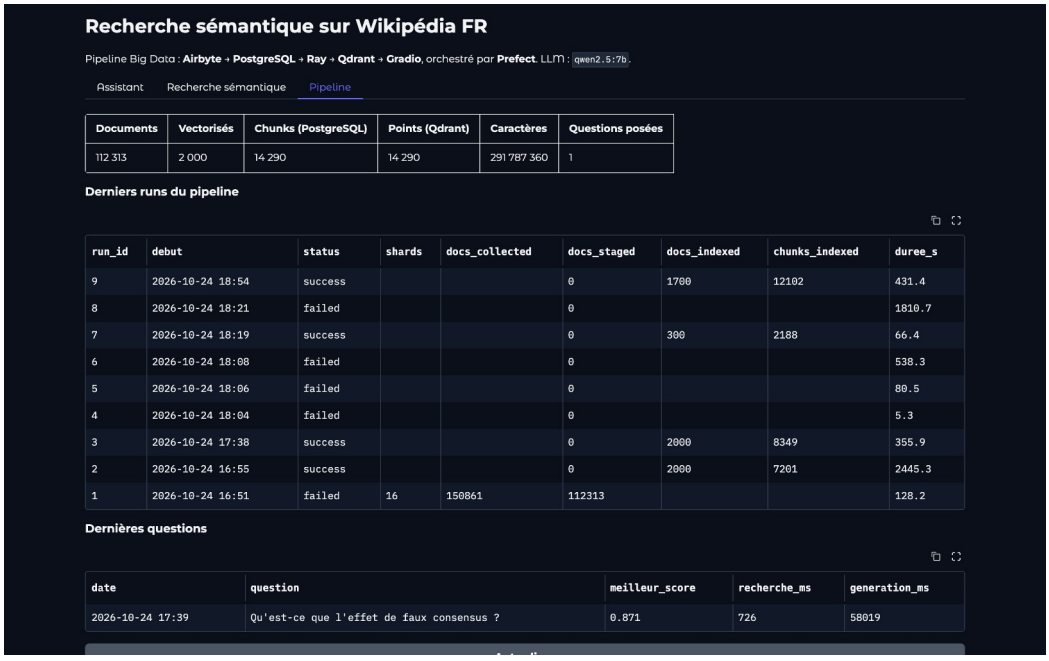


Figure 6 : onglet Pipeline : volumes, runs et questions

4.7 Génération de la réponse

Les passages retrouvés sont numérotés et envoyés au LLM avec la question. L'instruction système lui demande de répondre en français, **uniquement à partir des extraits**, de citer ses sources sous la forme [1], [2]… et de dire clairement quand les extraits ne permettent pas de répondre. La température est basse (0,2) pour limiter les inventions. La réponse est diffusée en continu (SSE), ce qui affiche les premiers mots après quelques secondes.

5. Résultats et performances

Toutes les mesures ont été prises sur un MacBook (10 cœurs), avec 8 Go de mémoire alloués à Docker, partagés avec d'autres conteneurs. Les durées proviennent de la table `pipeline_runs` et les mesures de qualité du script d'évaluation du dépôt (`src/ragpipe/evaluation/`).

5.1 Optimisation de la vectorisation

La vectorisation est l'étape la plus coûteuse. Trois runs successifs montrent l'effet des réglages du cluster Ray :

Run	Configuration	Articles	Passages	Durée	Débit
2	Configuration initiale : le head exécute aussi des tâches	2 000	7 201	2 445 s	2,9 passages/s
3	Head coordinateur seul, 3 CPU par worker, lecture et découpage fusionnés	2 000	8 349	356 s	23,5 passages/s
9	Passages de 500 caractères, lots de 32, attente des workers	1 700	12 102	431 s	28,1 passages/s

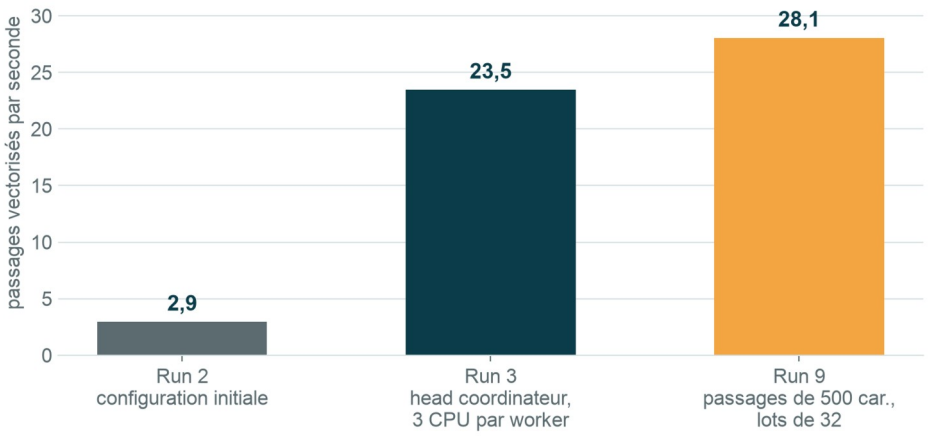


Figure 7 : *débit de vectorisation selon la configuration du cluster Ray (1 worker)*

Le premier run était lent parce que le nœud head exécutait lui aussi des tâches, et que le moniteur mémoire de Ray a tué la tâche d'écriture 41 fois. En réservant le head à la coordination, en donnant 3 CPU à chaque worker et en fusionnant lecture et découpage, le débit a été multiplié par 8. Les runs 2 et 3 utilisaient encore des passages de 1 000 caractères ; le run 9 utilise la taille définitive de 500 caractères, qui produit plus de passages par article.

Avec 1 worker, le débit est de 28 passages par seconde. Le fichier 16 complet (environ 800 000 passages) demanderait donc environ 8 heures, et le corpus entier plus d'une semaine. Le calcul des embeddings étant indépendant d'un passage à l'autre, ajouter des workers devrait augmenter le débit presque proportionnellement, mais nous n'avons pas pu le mesurer : avec 8 Go de mémoire Docker, un deuxième worker provoquait un manque de mémoire de la machine virtuelle (voir partie 6).

5.2 Qualité de la recherche

Méthode

Nous avons tiré au hasard 60 passages de l'index, un par article, avec une graine fixe pour que le tirage soit reproductible. Pour chacun, le LLM a écrit une question à laquelle ce passage répond, sans recopier ses phrases. On envoie ensuite chaque question au moteur de recherche (10 passages) et on regarde à quel rang apparaît l'article d'origine.

- **Hit@k** : part des questions dont l'article attendu figure parmi les k premiers passages.
- **MRR@10** (*Mean Reciprocal Rank*) : moyenne de 1/rang de l'article attendu (0 s'il est absent des 10 premiers). Vaut 1 si l'article est toujours premier.

Résultats

Mesure	Valeur
Hit@1 : article attendu en 1re position	86,7 %
Hit@5 : article attendu dans les 5 premiers	93,3 %
Hit@10 : article attendu dans les 10 premiers	95,0 %
MRR@10	0,899
Passage exact retrouvé dans les 5 premiers	70,0 %
Durée de la recherche : médiane / 95e centile	140 ms / 271 ms

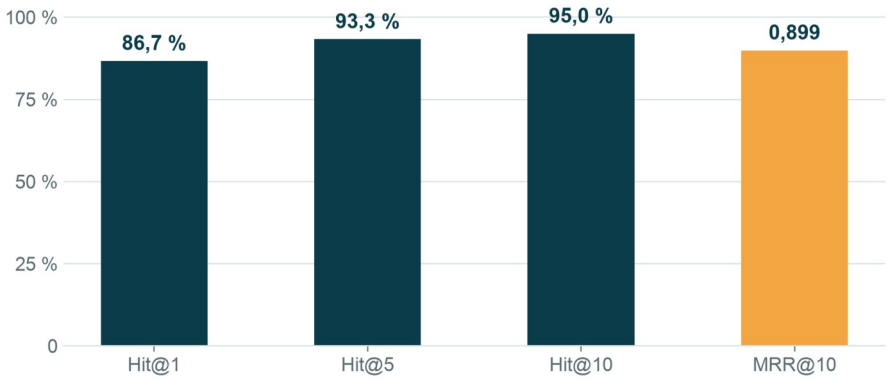


***Figure 8 :** qualité de la recherche sur 60 questions de test*

L'article attendu est retrouvé dans 57 cas sur 60, presque toujours en première position. Le passage exact qui a servi à écrire la question figure dans les cinq premiers résultats dans 70,0 % des cas : quand ce n'est pas lui, c'est souvent un autre passage du même article qui est remonté.

Les trois échecs concernent des questions qui ne nomment pas leur sujet. Par exemple, « Quel(s) pays a remporté une médaille masculine aux Championnats d'Europe de saut en hauteur en 2015 ? » a été écrite à partir de l'article sur un athlète, sans le citer : la recherche renvoie alors des articles sur d'autres championnats d'Europe.

Ces résultats sont **optimistes**. Les questions sont générées à partir des passages eux-mêmes, donc proches de leur vocabulaire, et l'index ne contient que 14 290 passages : dans un index de plusieurs millions de passages, les confusions entre articles voisins seront plus fréquentes. Un jeu de questions écrites par des personnes serait l'étape suivante.

5.3 Temps de réponse

La recherche (vectorisation de la question et requête Qdrant) prend 140 ms en médiane. Sur 5 questions avec génération complète, le premier mot de la réponse arrive après 4,72 s et la réponse complète après 7,9 s (médianes). Les 5 réponses citent leurs sources. La toute première question après le démarrage a été plus lente (58 s), probablement parce qu'Ollama doit d'abord charger le modèle en mémoire.

Question de test	1er mot	Total	Citations
Quel type de navire était Anna-Rosa et à quel usage servait-il durant de nombreuses années ?	4,06 s	6,98 s	oui
Quel type de transport est caractérisé par une longueur de Froude faible ?	4,72 s	7,9 s	oui
Quel partenaire a proposé à la WTC d'organiser la nouvelle épreuve de triathlon après la non-renouvellement de la licence d'Ironman Europe à Roth ?	4,66 s	10,84 s	oui
Quel année a été introduit le suivi de partition séparément par Barry Vercoe et Roger Dannenberg ?	5,8 s	6,18 s	oui
Quel(s) pays(s) a remporté une médaille masculine aux Championnats d'Europe de saut en hauteur en 2015 ?	5,72 s	11,33 s	oui

6. Difficultés rencontrées et solutions

Plusieurs problèmes sont apparus en conditions réelles. Chacun a été diagnostiqué puis corrigé dans le code ou la configuration.

Problème	Cause	Solution
Le connecteur Airbyte ne s'installe pas	Une de ses dépendances n'est pas compatible avec Python 3.12	Connecteur installé dans un environnement Python 3.11 isolé, préinstallé dans l'image
Run 2 très lent (2 445 s)	Le head exécutait des tâches ; le moniteur mémoire de Ray a tué l'écriture 41 fois	Head réservé à la coordination, 3 CPU par worker, lecture et découpage fusionnés : 356 s
Ray tue des tâches à tort, ou refuse de démarrer	Ray mesure la mémoire de toute la machine virtuelle Docker, pas celle du conteneur	Limite mémoire par conteneur, mémoire de Ray fixée au démarrage, redémarrage automatique
La moitié de chaque passage ignorée par le modèle	MiniLM ne lit que 128 tokens ; un passage de 1 000 caractères en compte environ 250	Passages de 500 caractères (75 de recouvrement), index reconstruit
Run 8 bloqué, puis en échec après 30 min	Manque de mémoire de toute la machine virtuelle Docker (8 Go partagés), repéré dans `memory.events`	Nombre de workers et d'acteurs paramétrable (`RAY_WORKERS`, `EMBED_ACTORS`), 1 par défaut
Erreur de division par zéro dans Ray Data	Agrégation distribuée lancée avant que les workers aient rejoint le cluster	Le job attend les workers, et compte les passages en lisant le flux
Historique de conversation perdu dans Gradio	Gradio 6 a changé le format des messages	Historique conservé dans un état propre à l'application (`gr.State`)

Le diagnostic du run 8 mérite d'être détaillé. Les compteurs du conteneur indiquaient des processus tués (`oom_kill`) alors que le conteneur n'avait jamais atteint sa propre limite (`oom = 0`, pic à 1,4 Go). C'est donc la machine virtuelle Docker entière qui manquait de mémoire, partagée avec un autre projet. Ce genre de problème est typique du Big Data sur une seule machine : les ressources doivent être planifiées pour l'ensemble des services, pas service par service.

7. Limites et perspectives

7.1 Limites

- **Volume indexé.** 2 000 articles sur 112 313 sont vectorisés, soit moins de 0,1 % du corpus complet. Le pipeline sait traiter le reste, mais cela prend du temps sur une seule machine.
- **Passage à l'échelle non mesuré.** La mémoire disponible (8 Go pour Docker) n'a pas permis de comparer 1, 2 et 4 workers.
- **Évaluation limitée.** 60 questions synthétiques sur un petit index : les scores sont un ordre de grandeur, pas une mesure définitive.
- **Latence de génération.** Plusieurs secondes par réponse avec un modèle de 7 milliards de paramètres sur un ordinateur portable.
- **Sécurité.** L'application n'a pas d'authentification et les mots de passe de démonstration sont dans `.env.example` : elle est faite pour un usage local.

7.2 Perspectives

- Déployer le cluster Ray sur plusieurs machines (par exemple avec KubeRay sur Kubernetes) et mesurer le débit avec 1, 2, 4 et 8 workers.
- Calculer les embeddings sur GPU, et tester des modèles plus récents qui lisent des passages plus longs.
- Améliorer la recherche : réglage de l'index HNSW, recherche hybride (mots-clés et vecteurs), re-classement des résultats par un second modèle.
- Construire un jeu de questions écrites à la main et mesurer aussi la fidélité des réponses aux sources.
- Mettre à jour le corpus à partir des nouveaux dumps Wikipédia : l'empreinte md5 permet déjà de ne revectoriser que les articles modifiés.
- Ajouter d'autres sources avec Airbyte (HAL, arXiv…) et partager l'application en ligne (Hugging Face Spaces).

8. Organisation de l'équipe

Chaque membre est responsable d'une brique : il la fait évoluer, la documente et la présente à la soutenance. La cheffe de projet assure l'intégration et la cohérence de l'ensemble.

Membre	Responsabilité	Code concerné
Salma Abarkane	Cheffe de projet, architecture, intégration, rapport	`docker-compose.yml`, `README.md`
Rania Bouaroua	Collecte (Airbyte)	`src/ragpipe/ingestion/`
Salma Benmouma	Orchestration (Prefect)	`src/ragpipe/orchestration/`
Oumaima Gassi	Stockage relationnel (PostgreSQL)	`sql/`, `storage/postgres.py`, `storage/staging.py`
Imad Eddine El Khorassani	Stockage NoSQL (Qdrant)	`storage/qdrant_store.py`
Mohammed Yassine Benmeziane	Traitement distribué (Ray)	`src/ragpipe/processing/`
Anys Louzal	Visualisation et partage (Gradio)	`src/ragpipe/app/`
Mohamed Anouar Faress	Couche RAG et évaluation	`retrieval/`, `evaluation/`

Le code est partagé dans un dépôt Git privé. Chaque brique est développée sur sa propre branche et intégrée à `main` par une pull request relue par un autre membre. Les secrets (`.env`) ne sont jamais versionnés, et les versions des dépendances sont figées pour que tout le monde travaille dans le même environnement.

9. Conclusion

Le projet livre un pipeline RAG complet et fonctionnel, de la collecte des données brutes jusqu'à une interface qui répond aux questions en citant ses sources. Chacune des six technologies imposées y tient un rôle précis : Airbyte pour la collecte, PostgreSQL pour la donnée structurée et le suivi, Ray pour le calcul distribué, Qdrant pour la recherche vectorielle, Prefect pour l'orchestration et Gradio pour l'interface.

Les mesures montrent qu'un pipeline Big Data se joue autant dans les réglages que dans le code : la répartition des rôles dans le cluster Ray a multiplié le débit par huit, et la taille des passages a dû être adaptée au modèle d'embedding. La recherche retrouve le bon article dans la grande majorité des cas, en une fraction de seconde.

L'architecture est prête à passer à l'échelle : ajouter des workers Ray, des fichiers du corpus ou de nouvelles sources ne demande que de changer des paramètres. La prochaine étape est de la déployer sur plusieurs machines pour indexer le corpus complet et mesurer ce passage à l'échelle.

Annexes

A. Lancer le projet

Prérequis : Docker Desktop avec au moins 8 Go de mémoire, et Ollama avec le modèle `qwen2.5:7b`.

```
git clone https://github.com/Anyslouzal/projet-rag-bigdata.git
cd projet-rag-bigdata
cp .env.example .env
docker compose up -d --build
make run          # collecte du fichier 16 + vectorisation de 2 000 articles
make test         # tests unitaires
make eval         # évaluation de la recherche (docs/evaluation/resultats.json)
```

B. Interfaces

Interface	Adresse
Application Gradio	http://localhost:7860
Prefect (runs, planification)	http://localhost:4200
Tableau de bord Ray (cluster, jobs)	http://localhost:8265
Tableau de bord Qdrant	http://localhost:6333/dashboard
PostgreSQL	localhost:5432, utilisateur rag

C. Structure du dépôt

```
docker-compose.yml      les 7 services du projet
Dockerfile              image Python unique pour tous les services
requirements.lock       versions figées des dépendances
sql/init/               schéma PostgreSQL
src/ragpipe/
  ingestion/            collecte Airbyte
  storage/              PostgreSQL, staging SQL, Qdrant
  processing/           découpage, embeddings, job Ray
  retrieval/            recherche sémantique et appel au LLM
  orchestration/        flow et déploiement Prefect
  evaluation/           évaluation de la recherche et de la génération
  app/                  application Gradio
tests/                  tests unitaires
docs/                   rapport, présentation, figures, résultats d'évaluation
```